Bangladesh Army University of Science and Technology
Saidpur, Nilphamari

Smart Railway gate control System

Submitted By

Student Name	Student ID
1. Sad Ibna Forid	0802510205101020
2. Rezwan Ahmed Torabee	0802510205101024
3. Sohana Sinthia Rahman	0802510205101017
4. Tahmid Tazowar Sachhya	0802510205101004
5. Syed Sifat Al Sayeed	0802510205101041

Submitted To

Teacher Name	Designation
1. Saad Ahmed	Lecturer

SESSIONAL PROJECT REPORT

This Report is Presented in Partial Fulfillment of the course "**CSE 2102: Digital Logic Design Sessional"** in the Department of Computer Science and Engineering

June 2, 2026

DECLARATION

We hereby declare that this sessional project has been done by us under the supervision of **Saad Ahmed**, **Lecturer**, Department of Computer Science and Engineering (CSE), Bangladesh Army University of Science and Technology (BAUST). We also declare that neither this project nor any part of this project has been submitted elsewhere as sessional project.

Submitted To:

Saad Ahmed
Lecturer
Department of Computer Science and Engineering
Bangladesh Army University of Science and Technology

Submitted by

Sad Ibna Forid
0802510205101020
Dept. of CSE, BAUST

Rezwan Ahmed Torabee
0802510205101024
Dept. of CSE, BAUST

Syed Sifat Al Sayeed
0802510205101041
Dept. of CSE, BAUST

Tahmid Tazowar Sachhya
0802510205101004
Dept. of CSE, BAUST

Sohana Sinthia Rahman
0802510205101017
Dept. of CSE, BAUST

Table of Contents

Chapter 1

Introduction

This chapter details the foundational framework of the Sentinel Gate system, describing the problem statement, motivation, project goals, and market gap analysis.

1.1 Introduction

Railway infrastructure is a vital part of global mass transit. However, level crossings—where train tracks and vehicular roads intersect—remain highly hazardous. Historically, these crossings have relied on manual gatekeepers or simple mechanical timers. These older methods are vulnerable to human error, communication delays, and poor visibility, which can lead to serious accidents.

To address these safety concerns, this project introduces "Sentinel Gate," an automated railway crossing system powered by an ESP32 microcontroller. The system uses a Finite State Machine (FSM) to process real-world sensor data and safely manage the gate's behavior. By replacing manual operations with reliable sensor automation, the project ensures accurate synchronization between approaching trains and the road barriers.

road barriers.

1.2 Motivation

The primary motivation behind Sentinel Gate is the need to upgrade traditional, safety-critical railway crossings into autonomous, highly responsive systems. By combining standard sequential logic with modern embedded hardware, we can create a safe, multitasking system that eliminates delays in operation.

From an academic standpoint, this project offers valuable hands-on experience in hardware-software integration. It bridges the gap between theoretical digital logic—such as state variables and signal handling—and practical engineering skills, including real-time embedded programming and sensor calibration.

1.3 Objectives

To resolve the structural issues identified in traditional level crossings, the specific technical objectives of this project are defined as follows:

- To design a reliable, non-blocking Finite State Machine (FSM) on the ESP32 that controls system behavior through clear, predictable steps.
- To use two MFRC522 RFID readers over a shared SPI bus for secure, bidirectional train identification using unique UID tags.
- To develop an automated safety override system using dual HC-SR04 ultrasonic sensors to monitor the track area between the gates.
- To implement a rapid-response safety feature that instantly halts the gate closure and reopens it to 90 degrees if an obstacle is detected within a 7 cm critical zone.

1.4 Feasibility Study

A thorough technical and operational assessment confirms that the Sentinel Gate architecture is highly viable for real-world use:

- **Technical Feasibility:** The ESP32 is equipped with a capable processor and dedicated hardware timers. This allows the system to simultaneously read sensor data and control servo motors precisely without freezing or delaying the main program.
- **Economic Feasibility:** The chosen components—such as the RFID readers, ultrasonic sensors, and buzzers—are affordable and energy-efficient. They provide excellent accuracy for this application at a fraction of the cost of industrial tracking systems.
- **Operational Feasibility:** The system runs on standard 5V and 3.3V DC power. It can easily operate on external battery packs or solar power, making it highly suitable for remote railway crossings that lack reliable access to the main power grid.

1.5 Gap Analysis

The limitations of conventional railway gate networks, compared to our proposed framework, are outlined below:

- **Train Detection Accuracy:** Many existing systems rely on standard infrared (IR) or laser sensors, which can trigger false alarms due to dust, wildlife, or bad weather. Sentinel Gate uses RFID readers to validate specific train tags, effectively eliminating false triggers.
- **Bidirectional Tracking:** Traditional setups often struggle if a train changes direction on the track. Our system dynamically updates the entry and exit points in real-time, depending on which RFID reader detects the train first.
- **Active Safety Override:** Standard crossings often close based purely on a timer, even if a vehicle is trapped on the tracks. Our design continuously scans for obstacles and will instantly halt the closing sequence if it detects an object within the safety zone.

1.6 Project Outcome

The successful implementation of this project yielded the following outcomes:

- **Physical Hardware Outcome:** We built a fully functional, multi-sensor level crossing prototype. The system includes automated servo barriers that respond to authorized RFID tags, an active warning buzzer, a three-color LED traffic light system, and an ultrasonic obstacle detection module.
- **Analytical Software Outcome:** We developed stable, deadlock-free firmware based on an FSM structure. The software ensures safe transitions between states, provides a 3-second yellow warning light for drivers, continuously checks for obstacles, and functions entirely offline without needing an internet connection.

Chapter 2

Proposed System Architecture

This chapter maps out the comprehensive structural configuration of the hardware elements, their physical connections, and the state-machine logic architecture.

2.1 Requirement Hardware and Software

The system relies on the ESP32 DevKit V1 as its primary core, communicating over the SPI bus for RFID detection and utilizing dedicated PWM lines for servo control.

2.1.1 Overview

Component	ESP32 Pin Connection	Operational Role
MFRC522 RFID 1 (Entry)	SPI (SCK=18, MISO=19, MOSI=23), SDA=5, RST=22	Scans approaching train unique ID
MFRC522 RFID 2 (Exit)	SPI (SCK=18, MISO=19, MOSI=23), SDA=4, RST=22	Scans departing train unique ID to open gate
HC-SR04 Sonar 1 & 2	Sonar 1: Trig=13, Echo=14 | Sonar 2: Trig=25, Echo=26	Monitors obstacle clearance between gates (< 7cm)
Servo Motors 1 & 2	Servo 1: Pin 27 | Servo 2: Pin 32	Actuates physical crossing barriers (0°/90°)
Traffic Indicators	Red LED=33, Yellow LED=15, Green LED=16	Provides visible traffic status indicators
Acoustic Buzzer	Pin 17 (Active Buzzer)	Sounds warning pulses and critical emergency alarms

2.1.2 Circuit Diagram

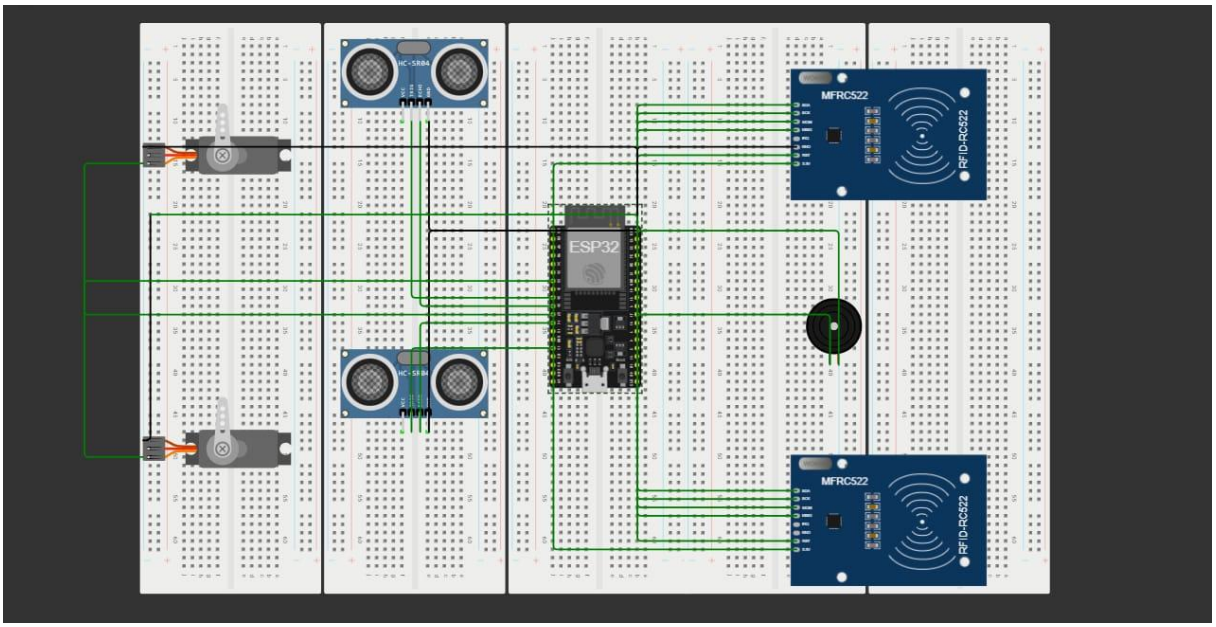
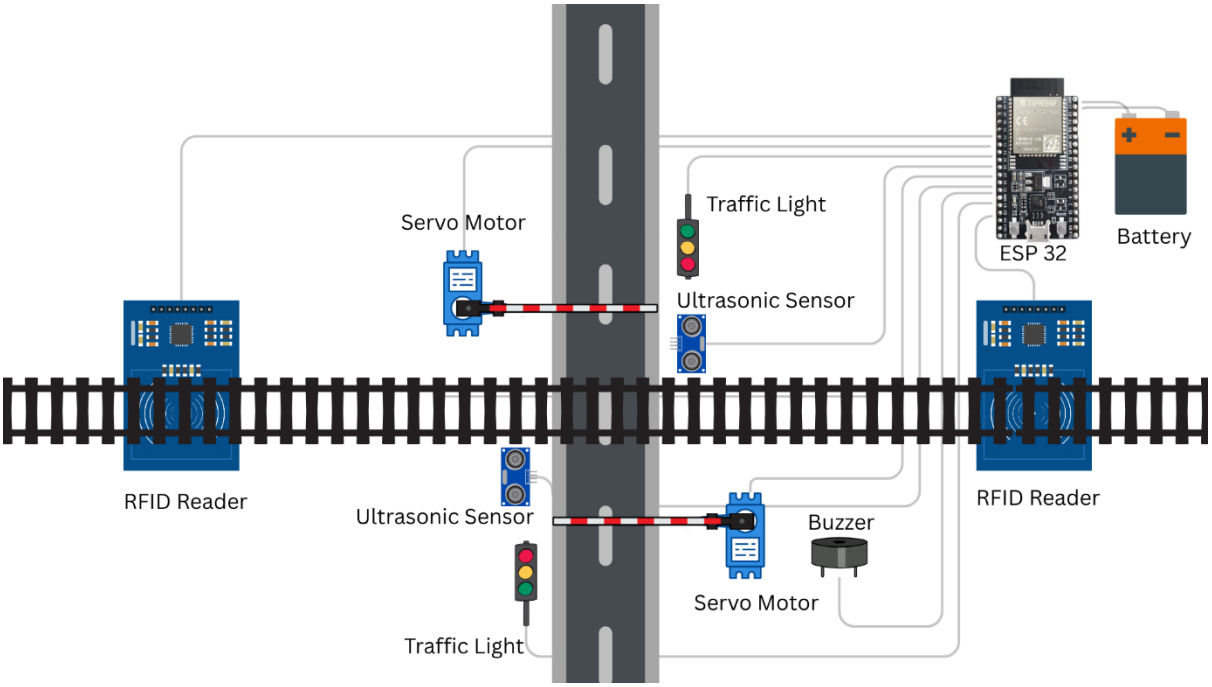


Figure 2.1: Circuit Diagram of Smart Railway gate control System

2.2 Project Implementation Plan

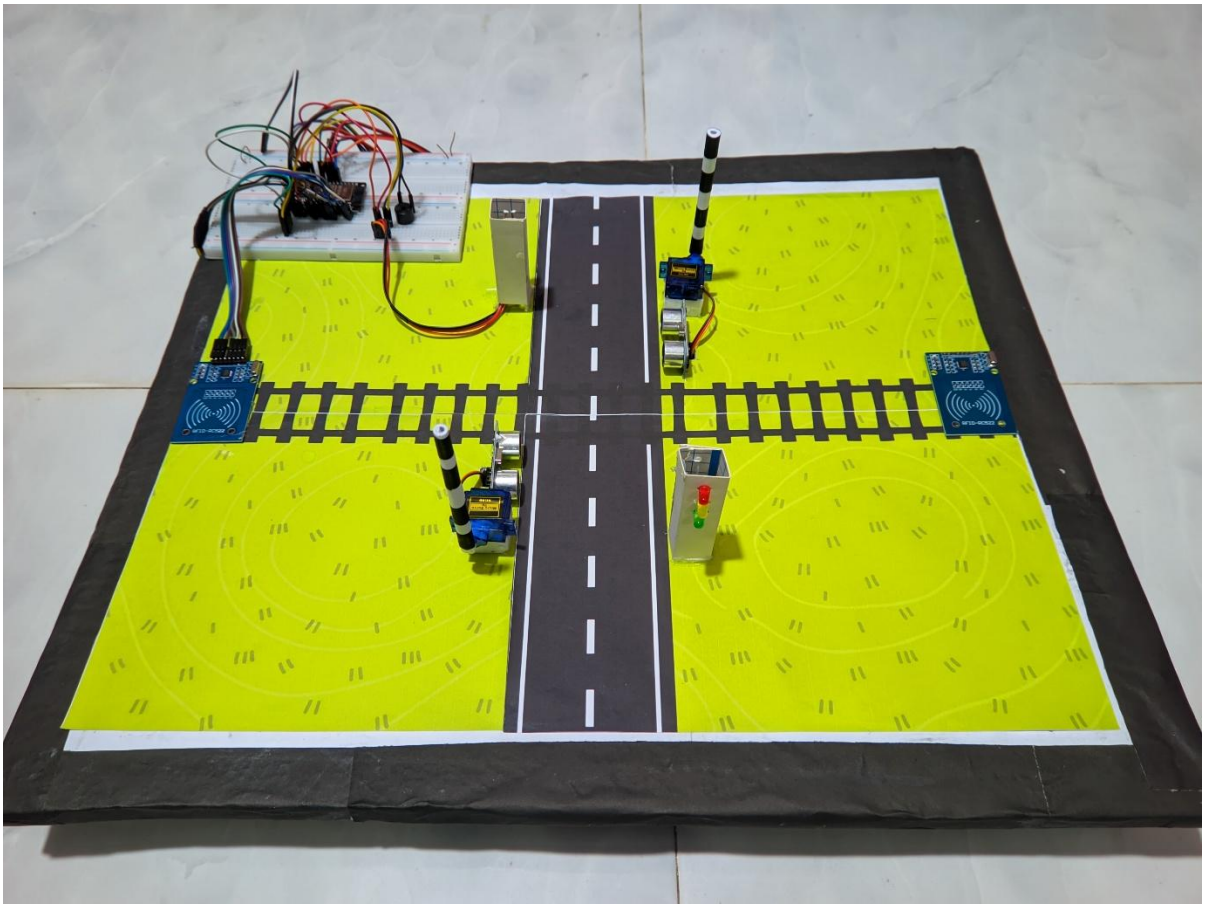


Chapter 3

Implementation and Results

This chapter displays the technical C++ source code utilized to drive the digital logic sequences and presents a detailed analysis of the real-time system behaviors. It focuses on the mapping of physical sensor variables into the software-driven Finite State Machine (FSM) and evaluates the output indicators under standard operational and emergency configurations.

3.1 Project Output Representation



3.2 ESP Code

```
#include <SPI.h>
#include <MFRC522.h>
#include <ESP32Servo.h>

// --- PIN DEFINITIONS ---
#define SCK_PIN 18
#define MOSI_PIN 23
#define MISO_PIN 19

// RFID 1 (Entry/Exit A)
```

```
#define SDA_1_PIN 5
#define RST_1_PIN 22

// RFID 2 (Entry/Exit B)
#define SDA_2_PIN 4
#define RST_2_PIN 21

// Sonar Sensors
#define TRIG1_PIN 13
#define ECHO1_PIN 14

#define TRIG2_PIN 25
#define ECHO2_PIN 26

// Servos
#define SERVO1_PIN 27
#define SERVO2_PIN 32

// Indicators
#define RED_LED 33
#define YELLOW_LED 15
#define GREEN_LED 16
#define BUZZER 17

// --- SETTINGS ---
const int GATE_OPEN_ANGLE = 90;
const int GATE_CLOSE_ANGLE = 0;
const int OBSTACLE_DISTANCE_CM = 7; // Trigger threshold for vehicle on
track

// --- OBJECTS ---
MFRC522 rfid1(SDA_1_PIN, RST_1_PIN);
MFRC522 rfid2(SDA_2_PIN, RST_2_PIN);
Servo gate1;
Servo gate2;

// --- UIDs ---
byte train1UID[] = {0xA4, 0xD7, 0xA4, 0xE5};
byte train2UID[] = {0x2A, 0x77, 0x02, 0xE1};

// --- SYSTEM STATES ---
enum SystemState {
  IDLE,
  WARNING_YELLOW,
  GATES_CLOSING,
  TRAIN_INSIDE,
  EMERGENCY_OBSTACLE,
  GATES_OPENING
};

SystemState currentState = IDLE;
int entryRFID = 0; // 1 if entered from RFID1, 2 if entered from RFID2
unsigned long stateTimer = 0;

void setup() {
  Serial.begin(115200);

  // Initialize SPI & RFIDs
  SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN);
  rfid1.PCD_Init();
  rfid2.PCD_Init();

  // Allow allocation of all timers for ESP32
  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);
  ESP32PWM::allocateTimer(2);
  ESP32PWM::allocateTimer(3);
```

```
  // Initialize Servos
  gate1.setPeriodHertz(50); // Standard 50hz servo
  gate2.setPeriodHertz(50);
  gate1.attach(SERVO1_PIN, 500, 2400);
  gate2.attach(SERVO2_PIN, 500, 2400);

  gate1.write(GATE_OPEN_ANGLE);
  gate2.write(GATE_OPEN_ANGLE);

  // Initialize IO
  pinMode(TRIG1_PIN, OUTPUT);
  pinMode(ECHO1_PIN, INPUT);
  pinMode(TRIG2_PIN, OUTPUT);
  pinMode(ECHO2_PIN, INPUT);

  pinMode(RED_LED, OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  setTrafficLights(LOW, LOW, HIGH); // Start with Green
  Serial.println("ESP32 Offline System Ready.");
}

void loop() {
  switch (currentState) {
    case IDLE:
      handleIdleState();
      break;
    case WARNING_YELLOW:
      handleWarningState();
      break;
    case GATES_CLOSING:
      handleGatesClosingState();
      break;
    case TRAIN_INSIDE:
      handleTrainInsideState();
      break;
    case EMERGENCY_OBSTACLE:
      handleEmergencyState();
      break;
    case GATES_OPENING:
      handleGatesOpeningState();
      break;
  }
}

// --- STATE HANDLERS ---

void handleIdleState() {
  int detectedReader = checkRFIDs();
  if (detectedReader > 0) {
    Serial.print("Train Detected at Reader: ");
    Serial.println(detectedReader);

    entryRFID = detectedReader;
    currentState = WARNING_YELLOW;
    stateTimer = millis();

    setTrafficLights(LOW, HIGH, LOW); // Yellow ON
    digitalWrite(BUZZER, HIGH); // Warning beep
    delay(200);
    digitalWrite(BUZZER, LOW);
  }
}

void handleWarningState() {
  // Wait 3 seconds for Yellow light before closing gates
```

```
  if (millis() - stateTimer > 3000) {
    currentState = GATES_CLOSING;
    setTrafficLights(HIGH, LOW, LOW); // Red ON
  }
}

void handleGatesClosingState() {
  gate1.write(GATE_CLOSE_ANGLE);
  gate2.write(GATE_CLOSE_ANGLE);

  // Check if a vehicle got trapped while gates are closing
  if (isObstacleDetected()) {
    triggerEmergency();
    return;
  }

  currentState = TRAIN_INSIDE;
}

void handleTrainInsideState() {
  // Continuously monitor for vehicles getting stuck under the gate
  if (isObstacleDetected()) {
    triggerEmergency();
    return;
  }

  // Check if train has reached the EXIT RFID
  int detectedReader = checkRFIDs();
  if (detectedReader > 0 && detectedReader != entryRFID) {
    Serial.println("Train exiting via opposite reader.");
    currentState = GATES_OPENING;
  }
}

void handleEmergencyState() {
  // Blink Red light and sound Buzzer continuously without blocking
  bool toggle = (millis() / 250) % 2;
  digitalWrite(RED_LED, toggle);
  digitalWrite(BUZZER, toggle);

  // Wait until obstacle clears
  if (!isObstacleDetected()) {
    delay(2000); // Give it 2 seconds of clear track to be sure
    if (!isObstacleDetected()) {
      Serial.println("Track clear. Resuming closure.");
      digitalWrite(BUZZER, LOW);
      setTrafficLights(HIGH, LOW, LOW); // Solid Red
      currentState = GATES_CLOSING;
    }
  }
}

void handleGatesOpeningState() {
  gate1.write(GATE_OPEN_ANGLE);
  gate2.write(GATE_OPEN_ANGLE);
  setTrafficLights(LOW, LOW, HIGH); // Green ON
  entryRFID = 0;
  currentState = IDLE;
}

void triggerEmergency() {
  Serial.println("EMERGENCY: Obstacle on track! Opening gates.");
  gate1.write(GATE_OPEN_ANGLE); // Open gates immediately to let vehicle out
  gate2.write(GATE_OPEN_ANGLE);
  currentState = EMERGENCY_OBSTACLE;
}

// --- HELPER FUNCTIONS ---
```

```
void setTrafficLights(bool red, bool yellow, bool green) {
  digitalWrite(RED_LED, red);
  digitalWrite(YELLOW_LED, yellow);
  digitalWrite(GREEN_LED, green);
}

int checkRFIDs() {
  // Check Reader 1
  if (rfid1.PICC_IsNewCardPresent() && rfid1.PICC_ReadCardSerial()) {
    if (isValidTrain(rfid1.uid.uidByte, rfid1.uid.size)) {
      rfid1.PICC_HaltA(); // Stop reading
      return 1;
    }
  }

  // Check Reader 2
  if (rfid2.PICC_IsNewCardPresent() && rfid2.PICC_ReadCardSerial()) {
    if (isValidTrain(rfid2.uid.uidByte, rfid2.uid.size)) {
      rfid2.PICC_HaltA(); // Stop reading
      return 2;
    }
  }

  return 0; // No valid train detected
}

bool isValidTrain(byte *uid, byte uidSize) {
  if (uidSize != 4) return false;

  bool isTrain1 = true, isTrain2 = true;
  for (int i = 0; i < 4; i++) {
    if (uid[i] != train1UID[i]) isTrain1 = false;
    if (uid[i] != train2UID[i]) isTrain2 = false;
  }
  return isTrain1 || isTrain2;
}

bool isObstacleDetected() {
  float dist1 = getDistance(TRIG1_PIN, ECHO1_PIN);
  float dist2 = getDistance(TRIG2_PIN, ECHO2_PIN);

  // Filter out zero readings (sensor timeouts)
  if (dist1 > 0 && dist1 < OBSTACLE_DISTANCE_CM) return true;
  if (dist2 > 0 && dist2 < OBSTACLE_DISTANCE_CM) return true;

  return false;
}

float getDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH, 30000); // 30ms timeout to prevent
blocking
  if (duration == 0) return 999.0; // Timeout

  return duration * 0.034 / 2;
}
```

3.3 Results and Discussion

Testing confirmed that the system behaves exactly as designed. When an authorized RFID tag is scanned at the entry point, the system immediately enters a pre-warning state, activating a yellow light for 3 seconds. If an object enters the 7 cm detection range of the ultrasonic sensors while the gates are closing (or are already closed), the system immediately switches to the EMERGENCY_OBSTACLE state. This forces the gates back open to 90 degrees and triggers a continuous audio buzzer and flashing red lights to warn nearby pedestrians and vehicles.

Chapter 4

Conclusion

This final chapter details the resource allocation across team members, functional constraints, and prospective system improvements.

4.1 Summary

The Railway Gate project successfully demonstrated the transition from theoretical digital logic to a fully functional embedded safety system. By integrating an ESP32 microcontroller with bidirectional RFID authentication and continuous ultrasonic monitoring, the prototype met all of its primary objectives. The system effectively modeled a non-blocking Finite State Machine (FSM) that processes real-time sensor data to manage gate actuation and emergency overrides. Ultimately, the project proved that localized, offline automation can significantly improve the safety and reliability of traditional level crossings without requiring complex or expensive infrastructure.

4.2 Limitation

Currently, the system is designed to operate entirely offline as a standalone unit. While this makes it immune to internet outages, it lacks remote monitoring capabilities. If an emergency occurs at the crossing, the local alarms will sound to warn those nearby, but central dispatchers or distant trains will not receive a remote digital alert.

4.3 Future Work

In future iterations, we plan to add independent solar charging circuits for sustainable power. We also aim to introduce a long-range communication module (such as Blynk or a dedicated IoT platform) to send instant alerts to remote operators and approaching trains if the crossing is blocked.